

Bringing Distributed Ordering to Heterogeneous Memory Consistency

Yanpeng Yu, *Yale University, New Haven, CT, 06520, USA*

Nicolai Oswald, *NVIDIA, Santa Clara, CA, 95051, USA*

Anurag Khandelwal, *Yale University, New Haven, CT, 06520, USA*

Abstract—CORD is a novel cache-coherence protocol that brings distributed-systems techniques to tightly coupled heterogeneous processors, unlocking efficient and programmer-friendly shared memory for emerging heterogeneous workloads such as AI/ML. Cache-coherence protocols have historically enforced consistency at the source processor, inflating application latency and interconnect traffic. Central to CORD's efficiency is its reframing of heterogeneous coherence as a distributed-systems problem: drawing on logical-timestamp algorithms for event ordering, CORD assigns consistency-tailored timestamps to write-through stores and orders them at the cache directory — near each store's completion point at the LLC. In our evaluation, CORD approaches message-passing efficiency while incurring a fraction of the programming complexity, with negligible storage, area, and power overheads. The result is a concrete demonstration that long-standing architectural challenges in shared-memory designs can be tackled through the lens of distributed systems.

The slowdown of Moore's Law and the end of Dennard Scaling have driven the rise of a rich ecosystem of heterogeneous processors — CPUs, GPUs, and beyond. Recent architectures couple these processors over cache-coherent fabrics — NVIDIA NVLink-C2C [1], AMD Infinity Fabric [2], Arm AMBA CHI [3], and CXL [4] — exposing programmer-familiar shared memory across heterogeneous processors for emerging workloads such as AI/ML and HPC. To accelerate producer–consumer communication, these fabrics adopt a *write-through* cache policy alongside the traditional write-back policy and expose *heterogeneous memory models* [5]. Among such models, *release consistency* has emerged as the de facto convergence point in production heterogeneous systems and is the focus of this paper.

Today's heterogeneous coherence protocols enforce consistency models at the *source processor* (e.g., AMBA CHI's Ordered Write Observation, CXL 3.0's UIO write completions): in release consistency, for instance, a Release store cannot issue until the cache directory acknowledges every prior write-through access in program order. We call this *source ordering* (Fig. 1, left). The acknowledgments inflate end-to-end application latency and interconnect traffic, particularly under the long latency and constrained bandwidth

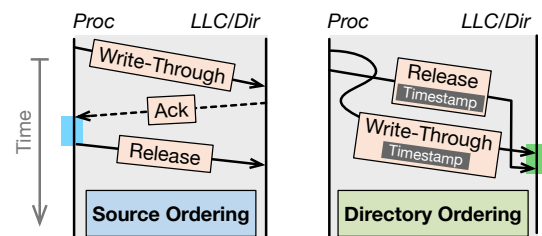


FIGURE 1. Unlike source ordering (left), CORD (right) orders write-through stores at the cache directory using logical timestamps.

of inter-processor fabrics that today's heterogeneous systems are built around. Message-passing protocols such as PCIe [6] avoid acknowledgments by relying on order-preserving network fabrics, but do so only end-to-end, breaking system-wide consistency properties and requiring more manual orchestration of ordering and synchronization.

We present CORD (**C**onsistency **O**rdered at **D**irectory), which resolves both shortcomings by rethinking heterogeneous coherence through the lens of distributed systems. Drawing on logical-timestamp algorithms [7], [8] for event ordering, CORD introduces *directory ordering*: it assigns consistency-tailored timestamps to write-through stores and enforces ordering at the cache directory — near each store's completion

point at the LLC (Fig. 1, right). By eliminating acknowledgments and processor-side waiting, directory ordering preserves shared-memory programming semantics while approaching message-passing efficiency.

We model-check CORD with Murphi and evaluate it on gem5 across diverse benchmark suites. Compared to the state-of-the-art source-ordered protocol Spandex [9], CORD improves end-to-end performance by 28% and 20% and reduces interconnect traffic by 11% and 16% over CXL and Intel UPI, respectively, at $\leq 1.3\%$ storage, area, and power overheads, while approaching message-passing efficiency at a fraction of its programming complexity. Beyond directory ordering itself, CORD illustrates how distributed-systems techniques can address long-standing architectural challenges in tightly coupled heterogeneous systems — a lens we develop further in *Discussion & Impact*.

Trends in Heterogeneous Systems

Write-through Cache-coherence

Recent heterogeneous systems [1], [2], [4] increasingly adopt cache-coherent shared memory across processors — e.g., NVIDIA's Grace Hopper [1] and future-generation Vera Rubin [10] Superchips enable direct memory access between CPUs and GPUs; CXL [4] extends coherence to rack-scale heterogeneous fabrics.

Importantly, modern heterogeneous coherence protocols [3], [4], [9] support a *write-through* (or write-combining) policy that propagates stores *directly* to the last-level cache or main memory — e.g., AMBA CHI's *WriteUnique/WriteNoSnp* [3] and CXL 3.0's Unordered IO (UIO) writes [4], as opposed to the traditional write-back policy. This is particularly useful in accelerating producer–consumer communication common in emerging heterogeneous workloads such as ML training and inference.

Release Consistency

Heterogeneous systems expose memory-level parallelism through *weak memory models* [5], which permit aggressive memory-access reordering. Across this design space, *release consistency* and its variants have emerged as the de facto choice [5], balancing performance with programmability. At a high level, release consistency lets memory accesses or fences be annotated with Acquire, Release, Acquire-Release, or Relaxed labels. A Release store, atomic, or fence acts as a barrier: prior accesses (in program order) cannot be reordered past it. Symmetrically, an Acquire load, atomic, or fence is a barrier that subsequent accesses

cannot be hoisted before. Acquire-Release accesses serve as full memory barriers, and Relaxed accesses carry no ordering constraints.

Need for Efficient Coherence

Inefficiencies in Memory Ordering

Coherence-protocol designers have long traded performance for design and verification simplicity — a sensible bargain in single-processor shared memory, where short on-chip latency and abundant bandwidth masked the resulting overheads. Heterogeneous systems can no longer make this trade-off: inter-processor latencies are long, interconnect bandwidths are constrained, and what was once a tolerable inefficiency now sits squarely on the critical path.

Memory ordering is arguably the most critical aspect where this tradeoff must be re-evaluated. Today's cache-coherence protocols enforce memory models at the source — the processor issuing the memory operation — even for write-through accesses completed at the last-level cache (LLC); under release-consistency semantics, for instance, a Release store cannot be issued until the cache directory has acknowledged all prior write-through accesses in program order. Production specifications such as AMBA CHI's Ordered Write Observation [3] and CXL 3.0 CXL.io's UIO write completions [4] implement this source-based mechanism — which we call *source ordering* — via such acknowledgments. The resulting split between ordering at the source and completion at the destination directory means every access incurs an acknowledgment round-trip and consumes bandwidth proportional to the communicated data volume, compounding into substantial application slowdowns and interconnect traffic inflation across heterogeneous workloads (over 30% in some cases, as we quantify later).

Why Not Message Passing?

To bypass source ordering's overheads, a common practice is to scale heterogeneous systems with message-passing protocols (e.g., PCIe [6] and NVLink), in which a processor directly modifies another's memory through “posted” transactions without waiting for acknowledgments.

Message-passing semantics are defined by ordering rules preserved by the interconnect fabric — e.g., PCIe's *Relaxed Ordering* and *Strong Ordering* — analogous to shared memory's consistency models. However, these rules apply end-to-end only and are not enforced across all processors in the system, thereby breaking properties such as release consis-

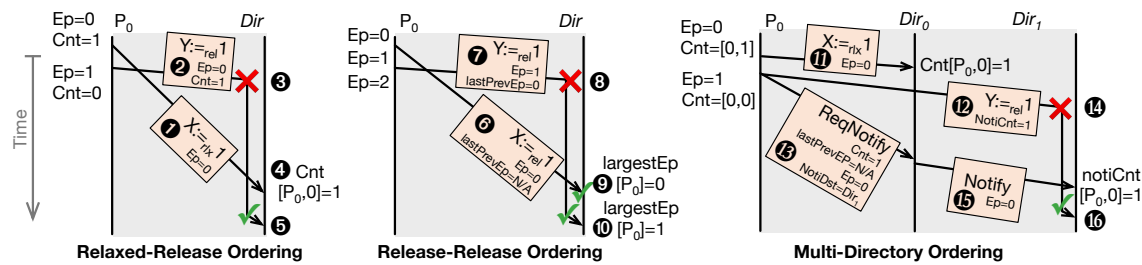


FIGURE 2. Examples of CORD's epoch number and store counter for relaxed-release ordering (left) and release-release ordering (middle), and CORD's inter-directory ordering for multi-directory release consistency (right).

tency's *synchronization cumulativity* [11]. Addressing this forces programmers to manually orchestrate inter-processor ordering and synchronization, making message passing materially harder to program than shared memory — a complexity gap that will only widen as heterogeneous systems embrace richer, more complex interconnect topologies.

CORD

CORD [12] rethinks heterogeneous cache coherence through the lens of distributed systems. The long interconnect latency and constrained bandwidth that characterize inter-processor communication in heterogeneous systems place modern shared memory in an operating regime closer to classical distributed systems — in contrast to the homogeneous on-chip setting for which traditional coherence protocols were designed. Drawing on logical-timestamp algorithms [7], [8] that order events across multiple nodes, CORD introduces *directory ordering*: rather than using the source processor as the ordering point and relying on acknowledgments, CORD assigns consistency-tailored timestamps to write-through requests and enforces ordering at the cache directory — i.e., near each request's completion point at the LLC or memory controller. By eliminating acknowledgment messages and processor-side waiting, directory ordering removes unnecessary interconnect round-trips and reduces coherence traffic while preserving shared-memory programming semantics.

Ordering for a Single-directory System

We begin by considering a simple system with a single shared directory. CORD's directory ordering decouples timestamps into two components: (i) an *epoch number* that divides logical time into periods between Release stores, and (ii) a *store counter* that tracks Relaxed stores within the current epoch and resets

on every Release store. CORD embeds the full timestamp (epoch and store counter) only in the infrequent Release store requests, and just the epoch number in the more frequent Relaxed store requests; the directory uses these embedded timestamps to commit write-through accesses in an order consistent with release consistency.

This decoupling sidesteps a fundamental trade-off that a consistency-agnostic timestamp design cannot escape. With one fixed-width timestamp per request, small bit-widths overflow frequently — forcing the processor to stall while prior timestamps drain and the counter resets — while large bit-widths inflate every request, incurring a high bandwidth overhead. CORD's timestamp decoupling achieves the best of both worlds: the small-bit-width epoch rides in existing transaction-packet headers (e.g., reserved bits), so Relaxed stores incur no bandwidth overhead, while the wide store counter travels only with the rare Release stores and overflows only for pathological workloads.

Enforcing Relaxed-Release Ordering. To prevent reorders between a prior Relaxed and a subsequent Release store at the same directory, each directory tracks a per-processor-epoch Relaxed store counter (Cnt[PID, Ep]). Fig. 2 (left) walks through the protocol: Relaxed stores embed the epoch (1) while Release stores embed both epoch and counter (2). On arrival, the directory commits a Relaxed store immediately and increments Cnt[PID, Ep] (4); a Release store commits only when its embedded counter matches Cnt[PID, Ep] (5), otherwise stalling (3).

Enforcing Release-Release Ordering. To prevent reorders between two Release stores at the same directory, each Release store embeds the last prior unacknowledged epoch (lastPrevEp; 6, 7) and each directory tracks the largest committed epoch per processor (largestEp[PID]). Fig. 2 (middle) shows the protocol: a Release store commits only when its em-

bedded lastPrevEp has been committed (9, 10), otherwise stalling (8).

Ordering across Multiple Directories

To extend directory ordering to real-world systems with multiple directories, CORD introduces an inter-directory notification mechanism: directories directly notify one another upon store completion, thereby enforcing cross-directory ordering without involving the source processor.

This eliminates the processor's wait for write-through acknowledgments and shortens the Release store's interconnect path, yielding lower processor stall *and* lower store latency than source ordering. Inter-directory notification can introduce additional control messages when a processor coordinates with many pending directories at once; in practice, real heterogeneous workloads keep fan-out small and synchronize at kilobyte granularity, so notification traffic still remains far below the acknowledgment traffic for source ordering, as we evaluate later.

Inter-directory notifications (Fig. 2 (right)). Beyond the timestamps used at a single directory, each Release store also embeds a notification counter (NotiCnt; 12) recording how many *pending* directories — those with uncommitted Relaxed or unacknowledged Release stores — the processor must coordinate with. The processor concurrently sends a request-for-notification (13) to each pending directory, which then sends a notification to the destination directory (15) once it commits its pending stores. The directory only commits the Release store when its received notifications match the embedded NotiCnt (16).

Bounding Storage Overhead

Fig. 3 (left) shows the micro-architectural data structures that CORD maintains at the processor and directory. All structures are implemented as look-up tables (with the per-processor epoch number realized as a single counter), and directory storage is statically partitioned per processor core. Per-epoch entries are reclaimed once the epoch is acknowledged at the processor or committed/notified at the directory, so storage does not accumulate indefinitely.

However, even with reclamation, in the worst case, the per-epoch tables still grow with the number of epochs, which can be prohibitively large at any reasonable bit width. Such worst-case scenarios are *extremely* rare in practice: they require an exact reversal of order across consecutive Release stores from the same processor, a pattern that did not occur even

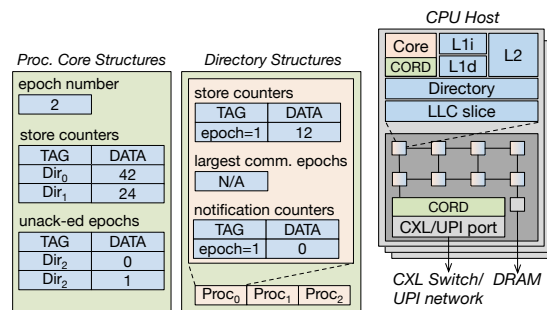


FIGURE 3. Micro-architectural implementation of CORD data structures at the processor core and directory (left), and simulated system architecture (right).

for synthetic worst-case benchmarks in our evaluation. CORD therefore provisions storage sufficient for the common case and falls back to *stalling* for correctness: before issuing a Release store, the processor checks that both its local unacknowledged-epoch table and the destination directory's store-counter and notification-counter tables have available entries, stalling the Release store if either lacks space until pending Release stores complete and free entries.

Composability with Existing Protocols

CORD's directory ordering applies only to write-through stores; other accesses — write-back stores, loads, memory barriers, and instructions with address, data, or control dependencies — are still source-ordered. When a directory-ordered write-through store must be ordered before a subsequent source-ordered access, the processor bridges the two ordering domains by injecting an empty directory-ordered Release barrier and stalling until it is acknowledged.

Correctness Verification

We model-checked CORD using Murphi, running 122 Armv8 release-consistency litmus tests generated by the herd tool and 180 custom litmus tests targeting corner cases (mixed CORD and source-ordered cores, under-provisioned look-up tables, and epoch/counter overflows). All tests passed, establishing CORD's release-consistent semantics and deadlock-freedom.

Evaluation Highlights

Methodology

Simulated architecture. Our simulated system models the hardware-based coherent memory realization

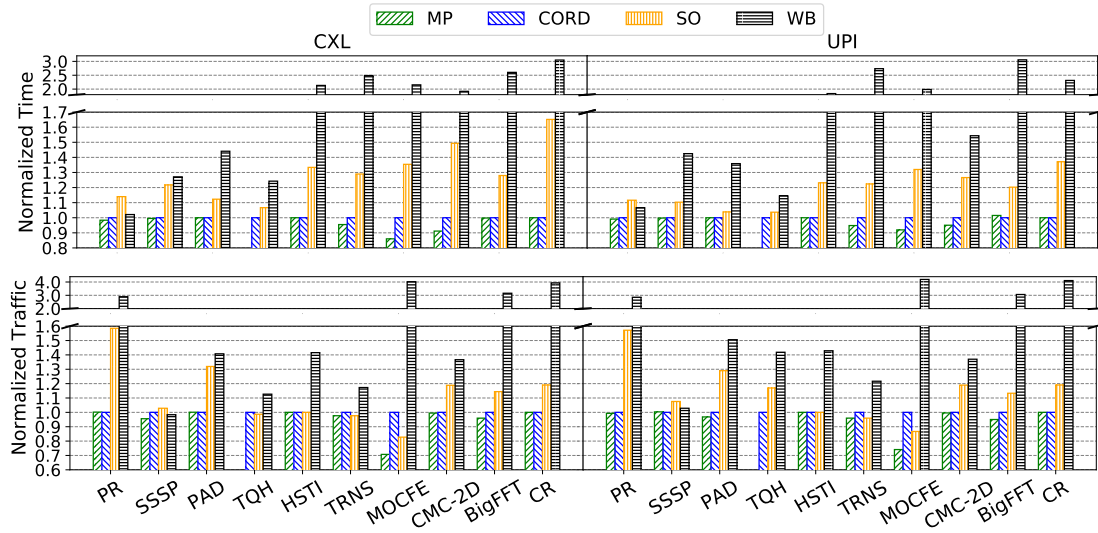


FIGURE 4. Performance (top) and traffic (bottom) for CORD, source ordering (SO), message passing (MP), and source ordering with write-back policy (WB) for end-to-end workloads. CORD outperforms SO by 28% and 20%, and comes within 4% and 2% of MP performance for CXL and UPI, respectively. CORD reduces SO traffic by 11% and 16%, and comes within 7% and 5% of MP traffic for CXL and UPI, respectively. Y-axes are normalized to CORD.

across multiple hosts described in the CXL 3.0 specification [4] as shown in Fig. 3 (right). It comprises multiple CPU hosts connected via a CXL switch. Each CPU host consists of a mesh of cores. Each core employs a core-private L1 data and instruction cache, a core-private L2 cache, and a slice of shared Last-level cache (LLC) co-located with its cache directory. Our simulation models CXL performance with an optimistic ~ 150 ns round-trip latency between two hosts, reflecting a lower bound on CORD's benefits since they grow with interconnect latency. To evaluate CORD on interconnects with shorter latency, we also model Intel UPI latency with the same system setup.

Workloads. We evaluate three benchmark suites with diverse write-through and synchronization patterns: Pannotia and Chai heterogeneous shared-memory benchmarks, and the U.S. Department of Energy (DOE) MPI mini-apps, message-passing workloads that we port to release-consistent shared memory.

Compared protocols. We compare CORD against three design schemes: (i) source-ordered write-through cache-coherence protocols (SO); (ii) message passing (MP); and (iii) source-ordered write-back cache-coherence protocols (WB)¹. For all com-

pared cache-coherence protocols, we use their corresponding MESI-based implementations provided by Spandex [9], a state-of-the-art heterogeneous cache-coherence protocol that supports flexible cache request interfaces. Our CORD implementation is also MESI-based. For MP, we simulate the PCIe protocol's read and write transactions with end-to-end ordering.

End-to-end Workloads

Performance. Fig. 4 (top) shows that CORD consistently outperforms SO across most workloads over both CXL and UPI. With CXL, CORD outperforms SO by over 28% on average, while with UPI, even though its benefits decrease, CORD still outperforms SO by over 20%. CORD observes significant performance benefits for the DOE workloads (i.e., 20–64% faster than SO for MOCFE, CMC-2D, BigFFT, and CR) because their communication-to-computation ratios are higher than other workloads.

Compared to MP, CORD maintains release consistency across all CPU hosts at less than 4% and 2% average performance overheads with CXL and UPI, respectively. Out of the seven workloads (TQH cannot run with MP), CORD performs more than 1% worse than MP only for TRNS, MOCFE, and CMC-2D. This is because their high communication fanout frequently triggers CORD's inter-directory notification mechanism, and their relatively fine-grained synchronization cannot

¹We refer to each scheme by its shorthand for the rest of this section.

offset the latency incurred by these notifications.

WB observes lower performance than CORD for all workloads except for `PR`. `PR` exhibits moderate locality, so that WB can improve performance with data reuse and employs relatively coarse-grained synchronization, subsuming the high latency of ordering WB stores at the source (same as SO).

Traffic. Fig. 4 (bottom) shows that CORD reduces the inter-processor traffic compared to SO across various workloads over both CXL and UPI. In particular, CORD reduces traffic compared to SO by over 11% and 16% for CXL and UPI, respectively. The largest savings appear on `PR`, `PAD`, `CMC-2D`, `BigFFT` and `CR` workloads (59% less traffic for `PR`, 11% – 29% for others). This is because they either employ fine Relaxed store granularity, making acknowledgment messages a significant traffic overhead (e.g., `PR` and `BigFFT`), or employ infrequent, coarse-grained synchronization (i.e., coarse Release granularity), making CORD's traffic overheads caused by the inter-directory notification messages small relative to the total traffic. In contrast, `TRNS` and `MOCFE` are the only workloads for which CORD generates more traffic than SO. This is because their fine-grained synchronization and high communication fanout trigger a high volume of inter-directory notifications, offsetting the traffic savings from eliminating write-through acknowledgments.

Compared to MP, CORD incurs 7% and 5% additional traffic on average over CXL and UPI, respectively. Specifically, out of the seven workloads for which MP can provide release consistency, CORD generates > 5% additional traffic only for two workloads (`PAD` and `MOCFE`) due to their fine-grained synchronization and medium to high communication fanout, as explained above.

WB generates significantly more traffic than CORD for all workloads except for `SSSP` because `SSSP` with its input graph `wing` is the only workload that simultaneously exhibits moderate locality so that WB can reduce traffic with data reuse, and employs relatively coarse-grained synchronization so that CORD's traffic saving from acknowledgments becomes insignificant.

Area and Power Overheads

For our 22 nm simulated configuration (8 cores per host with 8×2 MB shared LLC slices co-located with their directories), CACTI 7.0 estimates CORD adds 0.066 mm^2 / 9.24 mW per processor core and 0.136 mm^2 / 23.45 mW per directory (Table 1) — under 0.2% area and 1.3% static power of the per-host LLC + directory. Dynamic energy per look-up access is 0.016–0.025 nJ, an order of magnitude below the ~ 2 –

Component	Area (mm^2)	Power (mW)
CORD (per processor)	0.066	9.24
CORD (per directory)	0.136	23.45
LLC + directory (per host)	82.64	1761.26

TABLE 1. CORD's area and static-power overheads at 22 nm versus an LLC slice + directory baseline.

2.5 nJ to transmit a 64 B store on CXL 3.0 [4].

Discussion & Impact

Enforcing other consistency models

Heterogeneous memory models can be classified by whether they introduce *scopes* — thread cohorts within which consistency is enforced (e.g., PTX [5]) — or not (e.g., the compound memory model). For scoped memory models, CORD can be applied on a per-scope basis, with benefits growing with scope size: the larger the scope, the more processors and directories its synchronization spans, and the more source-ordered acknowledgments traverse the long inter-processor latency and constrained bandwidth that CORD targets. Consistency models also vary in ordering rules beyond scoping (e.g., x86-TSO); we present a TSO case study in [12].

Impact on single-processor coherence

As single processors (e.g., GPUs) scale to larger core counts and more physically distributed memory (e.g., wafer-scale or multi-die designs), maintaining a unified, globally addressable shared memory incurs prohibitive latency overhead under existing coherence protocols — forcing a fallback on message-passing-based inter-core communication, which requires programmers to explicitly orchestrate send/receive, buffering, and ordering across distributed cores even within a single accelerator. CORD's directory ordering attacks this latency bottleneck head-on: by collapsing source-ordering acknowledgments into directory-side timestamps, it re-opens the design space for unified shared memory in many-core accelerators.

Simplifying software for emerging workloads

Emerging workloads such as agentic AI comprise diverse subtasks scattered across heterogeneous processors — token generation on GPUs, vector retrieval for retrieval-augmented generation (RAG) or code execution on CPUs, external API calls via NICs — with frequent, fine-grained data sharing among them. Today, with message passing as the de facto communication primitive, this sharing forces every layer of the software

stack (compilers, languages, runtimes, libraries) to hand-orchestrate explicit send/receive with high complexity. By closing shared memory's performance gap with message passing, CORD reopens shared-memory abstractions for these workloads, so common patterns — task queues, KV-cache pools, producer-consumer buffers — can be simply expressed with ordinary loads and stores.

Redesigning shared memory with distributed-systems techniques

CORD is one of the first to bring distributed-systems protocols to tightly coupled heterogeneous processors: its directory ordering is inspired by the extensively studied logical-timestamp algorithm [7] from the distributed-systems community, illustrating how cross-layer insights can resolve long-standing architectural challenges. We expect this lens to inspire further cross-layer designs.

For instance, memory access traffic management in heterogeneous systems closely resembles network traffic management in distributed systems: cache-coherent fabrics must simultaneously carry fundamentally different classes of memory traffic, including latency-critical synchronization (e.g., locks, atomics) and high-volume bulk data transfers (e.g., tensor reads/writes, streaming memory copies). These traffic classes contend for the fabric, resulting in performance collapse — an effect well studied in packet-switched networks. Decades of work in network congestion control and Quality of Service (QoS) suggest that such contention can be mitigated by exposing traffic semantics to the fabric and enforcing differentiated treatment.

Path to Deployment

CORD's architectural principles map naturally onto NVIDIA's closely coupled CPU-GPU systems, where efficient coherent data sharing depends on minimizing interconnect traffic and coherence-related overheads.

To reduce coherent CPU-GPU communication latency, NVIDIA's Vera Rubin platform [10] implements a CORD variant that enables coherent write-through behavior for selected CPU writes.

Conclusion

Emerging heterogeneous systems employ shared memory with write-through coherence. We have shown that source ordering — the de facto mechanism for enforcing memory models in such systems — inflates

application latency and interconnect traffic. We presented CORD, which rethinks heterogeneous coherence through the lens of distributed systems: drawing on logical-timestamp algorithms, it orders write-through accesses at the directory rather than at the source. Compared to source ordering, CORD improves performance by 28% and 20% and reduces traffic by 11% and 16% over CXL and UPI, respectively, at $\leq 1.3\%$ storage, area, and power overheads, with correctness model-checked using Murphi.

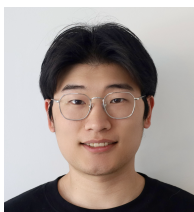
Acknowledgments

This article is an extension of a paper that appeared at ISCA 2025. We thank the anonymous ISCA reviewers for their valuable comments and insightful feedback. We would also like to thank David Nellans, Guillermo Juan Rozas, Gonzalo Brito Gadeschi, and Daniel Lustig for their feedback on various aspects of this work. This work was supported in part by the NSF awards 2047220, 2112562, 2147946, and a NetApp Faculty Fellowship.

REFERENCES

1. NVIDIA Corporation, "NVIDIA Grace CPU," in *Proc. IEEE Hot Chips Symposium*, 2022.
2. A. Smith, G. H. Loh, M. J. Schulte, M. Ignatowski, S. Naffziger, M. Mantor, N. Kalyanasundharam, V. Alla, N. Malaya, J. L. Greathouse, E. Chapman, and R. Swaminathan, "Realizing the AMD exascale heterogeneous processor vision: Industry product," in *Proc. ACM/IEEE ISCA*, 2024.
3. Arm Limited, "Arm AMBA CHI Architecture Specification, Issue H (IH0050)," [dohttps://developer.arm.com/documentation/ih0050/latest/](https://developer.arm.com/documentation/ih0050/latest/), 2024.
4. CXL Consortium, "Compute Express Link (CXL) Specification, Revision 3.0," [dohttps://www.computeexpresslink.org/download-the-specification](https://www.computeexpresslink.org/download-the-specification), Aug. 2022.
5. D. Lustig, S. Sahasrabudhe, and O. Giroux, "A formal analysis of the NVIDIA PTX memory consistency model," in *Proc. ACM ASPLOS*, 2019.
6. PCI-SIG, "PCI Express Base Specification, Revision 6.0," [dohttps://pcisig.com/specifications](https://pcisig.com/specifications), Jan. 2022.
7. L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
8. F. Mattern, "Virtual time and global states of distributed systems," in *Parallel and Distributed Algorithms*. North-Holland, 1989, pp. 215–226.

9. J. Alsop, M. Sinclair, and S. Adve, "Spandex: A flexible interface for efficient heterogeneous coherence," in *Proc. ACM/IEEE ISCA*, 2018.
10. NVIDIA Corporation, "NVIDIA Vera Rubin Platform," [dohttps://nvidianews.nvidia.com/news/nvidia-vera-rubin-platform](https://nvidianews.nvidia.com/news/nvidia-vera-rubin-platform), 2026.
11. S. Sarkar, P. Sewell, J. Alglave, L. Maranget, and D. Williams, "Understanding POWER multiprocessors," in *Proc. ACM PLDI*, 2011.
12. Y. Yu, N. Oswald, and A. Khandelwal, "CORD: Low-latency, bandwidth-efficient, and scalable release consistency via directory ordering," in *Proc. ACM/IEEE ISCA*, 2025.



Yanpeng Yu is a Ph.D. candidate in Computer Science at Yale University. His research focuses on computer architecture and systems, with an emphasis on cache-coherence protocols for large-scale shared memory systems. Contact him at yanpeng.yu@yale.edu.

yanpeng.yu@yale.edu.



Nicolai Oswald is a research scientist at NVIDIA, where he designs scalable architectures for LLM training and inference. His research focuses on aligning algorithms and hardware to improve the performance and efficiency of AI systems at scale. He received

his Ph.D. from the University of Edinburgh. Contact him at noswald@nvidia.com



Anurag Khandelwal is an Assistant Professor of Computer Science at Yale University. His research focuses broadly on problems in computer systems and networks. Anurag received his Ph.D. from the University of California, Berkeley in 2019. Contact

him at anurag.khandelwal@yale.edu.